wxcc-skills User Guide (draft)

Administer a Webex Contact Center tenant by talking to Claude.

This repo teaches Claude Code how to operate the WxCC Administration REST APIs through a library of *skills* — runbooks Claude loads automatically when your request matches one. You type plain English; Claude picks the skill, runs the verified API recipes inside it, and shows you the results.

> **Draft status:** this guide covers what exists today. Every API call documented in the skills was executed against a live tenant before it was written down — nothing is copy-pasted from docs on faith.

What you can do today

Ask things like:

You say	What happens
"How many users do we have? Who's on team X?"	Reads via the Users/Teams APIs
"Which entry point does +1 719 555 0100 route to?"	Dial-number → entry-point lookup
"Create a team called Billing on the Denver site"	Confirms with you, POSTs, verifies, tells you the rollback
"Move agent Jane to the Sales team"	Updates the user's team assignment (site/type rules enforced)
"Add a wrap-up code called Escalated"	Creates the auxiliary code
"What does the Default desktop profile let agents do?"	Reads the Desktop Profile config
"Raise queue X's service level threshold to 30 seconds"	Confirms, PUTs, re-reads to prove it changed
"How many calls did we take this week, by queue?"	Queries the GraphQL Search API over interaction data
"Send task-ended events to `https://my-server/hook`"	Registers a webhook subscription (confirms first)

Setup (once per machine, ~15 minutes)

Prerequisites: Python 3, Claude Code, a WxCC **administrator** account, and rights to create an app on developer.webex.com.

1. Clone this repo and open it in Claude Code.
2. Create an **Integration** at developer.webex.com with redirect URI `http://localhost:8484/callback` and scopes `cjp:config_read cjp:config cjp:config_write` (read-only? just `cjp:config_read`).
3. `cp .env.example .env` and fill in the client ID/secret and your region's API host.
4. Run `python wxcc.py auth login` — a browser opens; sign in as the admin and approve.
5. Check: `python wxcc.py auth status` should show `valid` plus the granted scopes.

Full walkthrough with troubleshooting: the **wxcc-connect** skill (.claude/skills/wxcc-connect/SKILL.md).

From then on, just ask Claude for things. Tokens refresh automatically (~14-day access token, ~90-day rolling refresh token).

The safety model

Reads and writes are treated differently, on purpose:

- **Reads are free.** List/inspect/search operations run without ceremony.
- **Writes always confirm first.** Before any create/update/delete, Claude states exactly what will change and waits for your yes.
- **Every write names its rollback** before it runs (create → delete the new id; update → put the captured original back; delete → flagged as effectively irreversible).
- **Every write is verified by a read.** This API can return 200 while silently ignoring a field — a lesson learned live — so "the API said OK" is never the proof; the re-read is.
- Credentials live in a gitignored `.env`; tokens in a gitignored `.wxcc/`. Nothing secret is ever committed.

Working with more than one tenant

If you administer several tenants, **there is deliberately no "switch tenant" command.** A mutable *current tenant* is how a delete meant for sandbox lands on production. Instead the tenant is chosen by the environment a process starts in, and it is visible in every call.

Give each tenant a **profile** — its own config file and its own token store, which never mix:

`WXCC_PROFILE`	config	tokens
(unset)	`.env`	`.wxcc/tokens.json`
`sandbox`	`.env.sandbox`	`.wxcc/tokens.sandbox.json`
`prod`	`.env.prod`	`.wxcc/tokens.prod.json`

A profile is only a label. The tenant is decided by whoever consents in the browser. Copying `.env` and a token file to a new profile name gets you a differently-named pointer to the *same* org — it does not move you to a new tenant. You must actually log in as an administrator of that tenant.

Authenticate each one once:

```
# bash / Git Bash
WXCC_PROFILE=prod python wxcc.py auth login
```

```
# PowerShell — there is no inline env-var prefix; set it, then run
$env:WXCC_PROFILE = "prod"
python wxcc.py auth login
python wxcc.py auth status        # verify the org id before trusting it
Remove-Item Env:WXCC_PROFILE      # clear it - don't leave a mode set in your shell
```

> ⚠️ **Sign out of Webex first, or use a private browser window.** If your browser is already signed in to another Webex identity, the consent screen will silently reuse that session and mint a token for the **wrong tenant** — and it will look like it worked.

Always confirm with `auth status` afterwards. It prints the profile *and* the resolved `org_id`. If the org id matches a tenant you already had, you authenticated as the wrong identity — run `auth logout` for that profile and try again in a clean browser session.

Two things that vary per tenant and are easy to miss:

- **Region.** `WXCC_API_BASE` differs by region (`us1`, `eu1`, `anz1`, …). Only `us1` has been exercised by this project.
- **The Integration.** One Integration's client ID usually works across orgs, but a corporate Control Hub can **block third-party integrations**. If consent fails or comes back with no CC scopes, register an Integration inside that tenant and give that profile its own client ID and secret.

Then register **one MCP server per tenant** in `.mcp.json` (copy `.mcp.json.example`; `.mcp.json` is gitignored, because server names tend to be real customer names). **Name each server the way you talk about the tenant** — `wxcc-acme`, not `wxcc-org2` — so that asking for "Acme's queues" routes itself:

```
{ "mcpServers": {
    "wxcc-sandbox": { "type": "stdio", "command": "python", "args": ["mcp_server.py"],
                      "env": { "WXCC_PROFILE": "sandbox" } },
    "wxcc-prod":    { "type": "stdio", "command": "python", "args": ["mcp_server.py"],
                      "env": { "WXCC_PROFILE": "prod" } } } }
```

Now the tenant is part of the tool name — `wxcc-sandbox` vs `wxcc-prod` — so Claude cannot silently act on the wrong one, and you can see which it used. Ask for "the sandbox queues" and it routes accordingly. `wxcc_whoami` reports the org id if you ever want to be sure.

Skill catalog

Skill	Covers	Writes?
wxcc-connect	OAuth setup, connectivity verification, troubleshooting	—
wxcc-users / wxcc-users-write	Find/inspect users; update team, site, profiles, CC enablement	update only (create/delete = Control Hub)
wxcc-teams / wxcc-teams-write	Teams	full CRUD
wxcc-queues / wxcc-queues-write	Contact service queues, routing groups, SLTs	full CRUD
wxcc-sites	Sites	read-only
wxcc-entry-points / wxcc-entry-points-write	Entry points + dial numbers (number↔EP mapping)	EP full CRUD; DN repoint (numbers themselves are provisioned in Calling)
wxcc-skill-profiles / wxcc-skill-profiles-write	Routing skills + skill profiles	full CRUD
wxcc-aux-codes	Idle + wrap-up codes	full CRUD
wxcc-address-books	Address books + entries	full CRUD
wxcc-outdial-ani	Outbound caller-ID lists	create/delete (update pending)
wxcc-desktop-profiles	Desktop Profiles (agent desktop behavior)	update (create/delete pending)
wxcc-desktop-layouts	Agent Desktop screen layouts (embedded layout JSON)	full CRUD
wxcc-tasks-search	Calls/tasks/agent sessions — reporting & wallboard data	read-only
wxcc-webhooks	Event subscriptions — push agent/task events to your server	full CRUD

> Naming note: Desktop Profiles appear in the API as `agent-profile` — that name is backwards compatibility only. Say "Desktop Profile."

How it works (one paragraph)

Every skill drives one shared helper, `wxcc.py` — a dependency-free Python CLI owning OAuth (authorization-code flow as your admin identity), token storage/refresh, org-id resolution, pagination, and

authenticated GET/POST/PUT/DELETE against your region's `api.wxcc-*.cisco.com` host. Skills are markdown runbooks with verified commands, field lists, and trap tables (the 404s, silent failures, and validation rules found by probing a live tenant). Longer term, the helper is intended to become an MCP server.

Known limits (honest list)

- User **creation/deletion** is not possible via this API (Control Hub owns identity); names/emails are read-only here.
- **Phone numbers cannot be invented**: dial-number records map numbers that already exist in the Webex Calling location inventory (provisioning stays in Control Hub).
- Sites are still read-only.
- **Flows are out of scope here, by design.** Cisco ships its own `flow-store` MCP server for flow authoring; run it alongside this one rather than expecting flow tools from us. What this repo gives you is the config the flows *bind to* — entry points, queues, teams, skill profiles.
- Bulk import/export is on the roadmap, not built.
- Region host is configured manually (`WXCC_API_BASE`); only us1 has been exercised.
- Anything a skill marks "candidate" has not been run against a live tenant yet.

Roadmap

Bulk-export · GraphQL aggregations (wallboard formulas) · site writes · remote (Cloud Run) MCP deployment · this guide, expanded.

Draft 2 — 2026-07-11. Feedback welcome: what would you ask it to do first?